

Sistemi Operativi

Corso di Laurea in Informatica

Sapienza Università di Roma

a.a. 2026/2027

Marco Raoul Marini

marini@di.uniroma1.it

Gabriele Tolomei

tolomei@di.uniroma1.it

Lecture 1.4: C Programming II

| Lecture Roadmap

"Every line of C you write becomes a machine instruction. Understanding the journey is understanding the craft."

8 major sections — from runtime memory to the build toolchain:

1. **The Stack** — frames, function calls, recursion
2. **The Heap** — dynamic allocation with `malloc` family
3. **Memory Bugs** — leaks, dangling pointers, double-free, UB
4. **Preprocessing** — macros, `#include`, conditional compilation
5. **Compilation & Assembly** — from C to machine code
6. **Linking** — symbol resolution, object files, executables
7. **Static vs Dynamic Linking** — `.a` vs `.so`, runtime loading

By the end, you will understand the complete lifecycle of a C program.

The Life of a C Program

Source code

hello.c
↓
cpp (preprocessor)
↓
hello.i (expanded source)
↓
cc1 (compiler)
↓
hello.s (assembly)
↓
as (assembler)
↓
hello.o (object file)
↓
ld (linker) + libc.a / libc.so
↓
hello (executable ELF)
↓
exec() system call
↓
Running process

Running process

Process in RAM:

Stack
Heap
.bss
.data
.text

Process Address Space

Every running C program sees a virtual address space organized in segments:

High addresses	argv, environ	command-line args
	STACK (grows downward ↓)	local vars, frames
	... gap ...	
	HEAP (grows upward ↑)	malloc'd memory
	.bss (uninitialized data)	zeroed globals/statics
Low addresses	.data (initialized data)	globals with values
	.text (code)	executable instructions

*Stack and heap grow toward each other. If they meet → **out of memory** / crash.*

§ 1 — The Stack

| What is the Stack?

The **stack** is a region of memory used for:

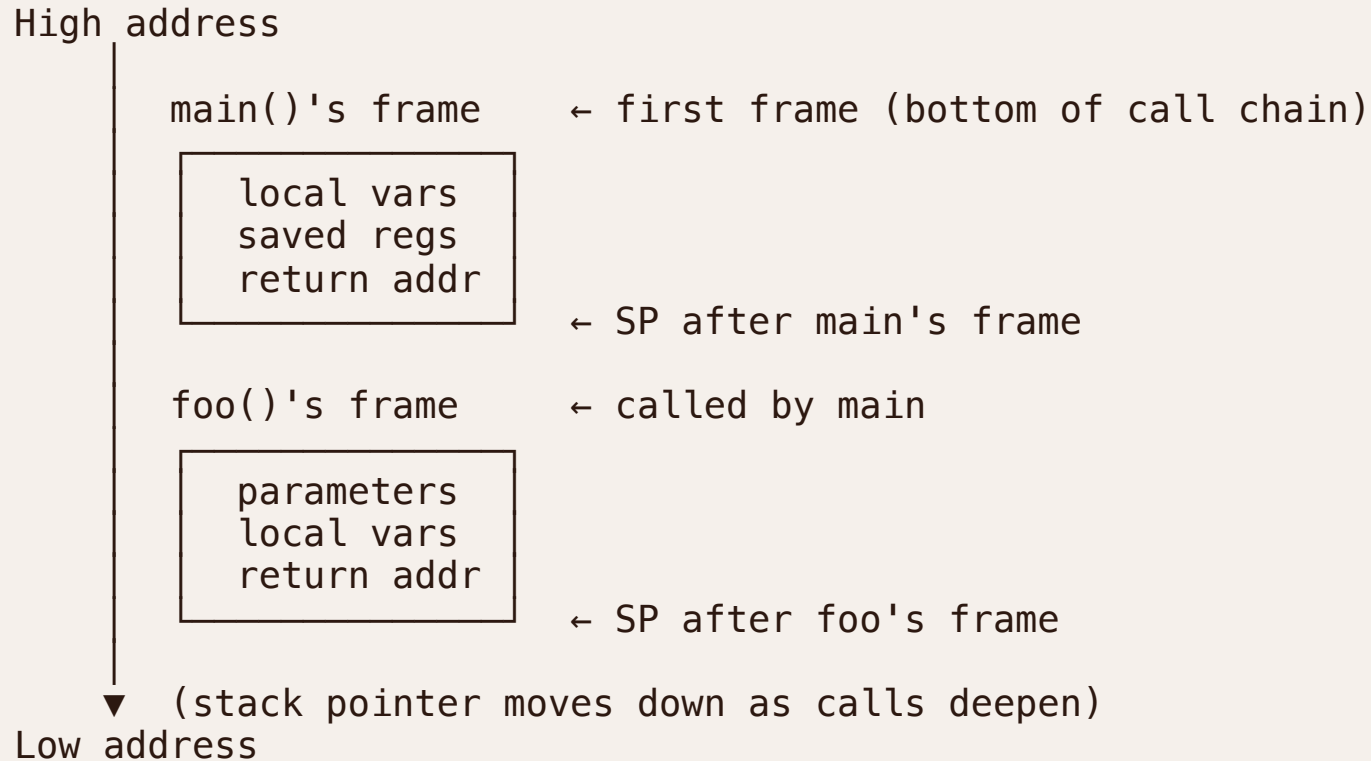
- **Local variables** — variables declared inside functions
- **Function parameters** — arguments passed to functions
- **Return addresses** — where to resume after a function returns
- **Saved registers** — CPU register state between calls
- **Return values** — small values (often via register, sometimes stack)

Key properties:

- **LIFO** (Last In, First Out) discipline
- **Automatic** management — allocated on call, freed on return
- **Fixed limit** — typically 1–8 MB per thread (OS-controlled)
- **Fast** — just a pointer adjustment to allocate/free

Stack Growth Direction

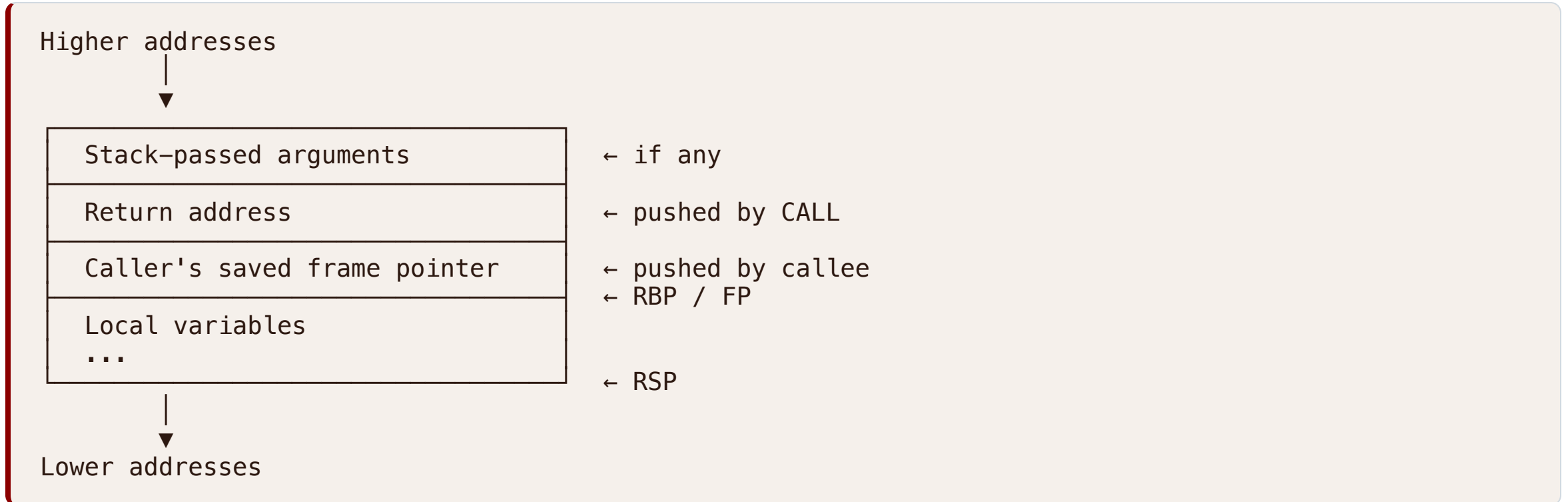
The stack grows **downward** on x86/x86-64 (toward lower addresses):



SP (Stack Pointer) always points to the top (lowest address) of the current stack.

Stack Frames (Activation Records)

Each function call creates a **stack frame** (also called an activation record). On x86-64, a conventional frame is built in this order:



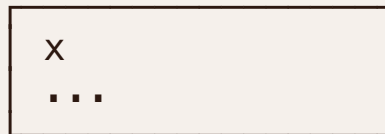
*The **Frame Pointer (FP/RBP)** provides a stable reference point within a frame, allowing debuggers to walk the call chain.*

Stack Frame Contents — Concrete Example

```
int add(int a, int b) {  
    int result = a + b;    // local variable  
    return result;  
}  
  
int main(void) {  
    int x = add(3, 7);    // call  
    return 0;  
}
```

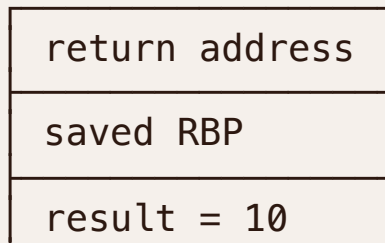
Stack during add(3, 7):

main's frame:



← local variable of main

add's frame:



← pushed by CALL

← pushed by callee (if using RBP)

← RBP

← local variable

Function Call — Step by Step

What happens at the hardware level when `add(3, 7)` is called:

1. CALLER prepares arguments:
 - On x86-64: first 6 integer args go in registers RDI, RSI, RDX, RCX, R8, R9
 - `add(3, 7)`: RDI=3, RSI=7
2. CALL instruction executes:
 - pushes return address onto stack (address of next instruction)
 - jumps to `add`'s first instruction
3. CALLEE prologue (`add`'s entry):

```
push rbp      ; save caller's frame pointer
mov  rbp, rsp ; set up new frame pointer
sub  rsp, N   ; allocate space for locals
```
4. Function body executes
5. CALLEE epilogue (`add`'s exit):

```
mov  rsp, rbp ; restore stack pointer
pop  rbp      ; restore frame pointer
ret                ; pop return address → jump back to caller
```

The actual parameters `a=3` and `b=7` are passed through registers. If the number of function parameters is greater than the available registers, the caller push them onto the stack.

Stack Registers: SP and FP

Two dedicated registers manage the stack on x86-64:

Register	Name	Role
RSP	Stack Pointer	Points to top of stack (lowest address in use)
RBP	Base/Frame Pointer	Points to base of current frame (stable reference)

RSP always tracks the current top:

`push rax` → `RSP -= 8`, then `[RSP] = RAX`

`pop rax` → `RAX = [RSP]`, then `RSP += 8`

RBP is anchored at frame start:

Local variable at offset -8 from RBP → `[RBP - 8]`

Saved caller RBP → `[RBP]`

Return address → `[RBP + 8]`

First stack argument → `[RBP + 16]`

| Calling Conventions

A **calling convention** is a contract between caller and callee about:

Aspect	x86-64 System V (Linux/macOS)
Integer args	RDI, RSI, RDX, RCX, R8, R9 (then stack)
Float args	XMM0–XMM7
Return value	RAX (integer), XMM0 (float)
Caller-saved	RAX, RCX, RDX, RSI, RDI, R8–R11
Callee-saved	RBX, RBP, R12–R15
Stack alignment	16-byte aligned before CALL

Caller-saved: callee may freely clobber these — caller must save if needed.

Callee-saved: callee must restore these before returning.

Stack Frame in C — Observed with Code

```
#include <stdio.h>

void show_addresses(int a, int b) {
    int local1 = 10;
    int local2 = 20;
    printf("&a          = %p\n", (void *)&a);
    printf("&b          = %p\n", (void *)&b);
    printf("&local1    = %p\n", (void *)&local1);
    printf("&local2    = %p\n", (void *)&local2);
}

int main(void) {
    int x = 5;
    printf("&x          = %p\n", (void *)&x);
    show_addresses(1, 2);
    return 0;
}

// Notice: addresses decrease as stack grows downward
// local1 and local2 are adjacent (4 bytes apart)
// x (in main) is at a HIGHER address than locals in show_addresses
```

Nested Function Calls

Each call adds a new frame; returns remove it:

Calling chain: main → foo → bar

High addr

main frame	x, y, saved regs
foo frame	a, b, result, return→main
bar frame	p, q, return→foo

Low addr ← SP

After bar returns:

main frame	
foo frame	← SP restored here

After foo returns:

main frame	← SP restored here
------------	--------------------

Recursion & the Stack

Every recursive call creates a **new frame**:

```
#include <stdio.h>

int factorial(int n) {
    printf("factorial(%d) called\n", n);
    if (n <= 1) return 1;
    return n * factorial(n - 1);    // new frame pushed for each call
}

int main(void) {
    printf("%d\n", factorial(5));    // 120
    return 0;
}

// Stack at deepest point (n=1):
// [main] → [fact(5)] → [fact(4)] → [fact(3)] → [fact(2)] → [fact(1)]
// 6 frames on stack simultaneously
// Each frame holds: n, return addr, saved FP
```

*Deep recursion = many frames = potential **stack overflow**. Prefer iteration for large depths.*

Stack Overflow

Stack overflow occurs when recursion (or large locals) **exhaust** the stack:

```
// Infinite recursion → stack overflow:
void infinite(void) {
    int big_array[10000];    // 40 KB per frame!
    infinite();              // each call adds 40 KB to stack
}
// After ~200 calls: SIGSEGV (segmentation fault)
// Default stack size on Linux: 8 MB

// Checking stack size:
// ulimit -s          → shows limit in KB
// ulimit -s unlimited → removes limit (careful!)

// Typical symptoms:
// - SIGSEGV with no obvious null pointer
// - Crash in deeply recursive code
// - Large local arrays in recursive functions
```

Use **heap allocation** (`malloc`) for large data in recursive functions.

Local Variables and Stack Lifetime

Local variables exist **only while their frame is alive**:

```
#include <stdio.h>

int *bad_pointer(void) {
    int local = 42;
    return &local;    // ⚠ returning address of local variable!
}                    // frame destroyed here → local no longer valid

int main(void) {
    int *p = bad_pointer();
    // p is now dangling – local's memory may be reused
    printf("%d\n", *p);    // ⚠ UB: may print 42, may crash, may print garbage

    // GCC will warn: -Wreturn-local-addr
    return 0;
}
```

Rule: Never return a pointer to a local variable. Use the heap if the data must outlive the function.

Returning Values from Functions

How return values flow back to the caller:

```
// Small values (int, pointer, float) → returned in registers:  
// RAX for integers/pointers, XMM0 for doubles  
  
int    square(int x)    { return x * x; }    // result in RAX  
double half(double x)  { return x / 2.0; }  // result in XMM0  
char *hello(void)      { return "hello"; }  // pointer in RAX  
  
// Large structs → caller allocates space, passes hidden pointer:  
typedef struct { int x, y, z, w; } Vec4;  
  
Vec4 make_vec(int a) {  
    Vec4 v = {a, a*2, a*3, a*4};  
    return v;    // copy into caller-provided space  
}  
// gcc may optimize this with RVO (Return Value Optimization)
```

✓ Checkpoint 1 — The Stack

Quick self-check:

1. What are the **four main things** stored in a stack frame?
2. In which direction does the stack grow on x86-64?
3. What is the role of the **Frame Pointer (RBP)**?
4. What happens to a local variable's memory after its function returns?
5. What causes a **stack overflow**, and how can you avoid it?

💬 Sketch a 3-level call stack for `main → foo(int) → bar(int, int)`.

§ 2 — The Heap

| What is the Heap?

The **heap** is a region of memory for **dynamic allocation** — memory whose size or lifetime is not known at compile time.

Key properties:

- **Manual management** — you allocate (`malloc`) and free (`free`) explicitly
- **Persists across function calls** — heap memory lives until you `free` it
- **Flexible size** — can allocate from a few bytes to gigabytes
- **Slower than stack** — involves OS/allocator bookkeeping
- **Shared** across all functions in the process

When to use the heap:

- ✓ Size not known at compile time
- ✓ Data must outlive the function that creates it
- ✓ Large allocations (arrays of thousands of elements)
- ✓ Data structures that grow/shrink at runtime (linked lists, trees)

Stack vs Heap – Comparison

Property	Stack	Heap
Management	Automatic (compiler)	Manual (<code>malloc</code> / <code>free</code>)
Size	Small (1–8 MB)	Large (GBs available)
Speed	Very fast	Slower
Lifetime	Until function returns	Until <code>free()</code> is called
Allocation	Push/pop (SP adjustment)	<code>malloc()</code> call
Fragmentation	None	Possible
Errors	Stack overflow	Leaks, double-free, dangling
Scope	Local to function	Global (any function can access)

Choose stack for small, short-lived data. Choose heap for large, long-lived, or dynamically-sized data.

malloc — Memory Allocation

`malloc(n)` allocates `n` bytes on the heap and returns a `void *`:

```
#include <stdlib.h>
#include <stdio.h>

int main(void) {
    int n = 10;

    // Allocate array of n ints:
    int *arr = malloc(n * sizeof(int));

    // ALWAYS check for allocation failure:
    if (arr == NULL) {
        fprintf(stderr, "malloc failed\n");
        return 1;
    }

    // Use the array:
    for (int i = 0; i < n; i++) {
        arr[i] = i * i;
        printf("%d ", arr[i]);
    }
    printf("\n");

    free(arr);    // release memory back to heap
    arr = NULL;   // prevent dangling pointer
    return 0;
}
```

calloc — Cleared Allocation

`calloc(n, size)` allocates `n` elements of `size` bytes each, **zero-initialized**:

```
#include <stdlib.h>
#include <stdio.h>

int main(void) {
    int rows = 3, cols = 4;

    // Allocate zero-initialized 2D flat array:
    int *matrix = calloc(rows * cols, sizeof(int));
    if (!matrix) { perror("calloc"); return 1; }

    // All elements are guaranteed to be 0:
    for (int i = 0; i < rows * cols; i++) {
        printf("%d ", matrix[i]);    // all zeros
    }

    free(matrix);
    return 0;
}

// malloc vs calloc:
// malloc(n * size)      → uninitialized (garbage values)
// calloc(n, size)       → zero-initialized (safer but slightly slower)
```

realloc – Resize Allocation

`realloc(ptr, new_size)` resizes a previously allocated block:

```
#include <stdlib.h>
#include <stdio.h>

int main(void) {
    int capacity = 4;
    int *arr = malloc(capacity * sizeof(int));
    if (!arr) return 1;

    arr[0]=1; arr[1]=2; arr[2]=3; arr[3]=4;

    // Need more space:
    capacity = 8;
    int *tmp = realloc(arr, capacity * sizeof(int));
    if (!tmp) {
        free(arr);          // ⚠ must free original on realloc failure
        return 1;
    }
    arr = tmp;              // realloc may move the block to a new address!

    arr[4]=5; arr[5]=6; arr[6]=7; arr[7]=8;
    free(arr);
    return 0;
}
```

free — Releasing Memory

`free(ptr)` returns heap memory to the allocator:

```
#include <stdlib.h>

int main(void) {
    int *p = malloc(sizeof(int) * 5);
    if (!p) return 1;

    // Use p...

    free(p);        // release the memory
    p = NULL;       // ✓ critical: null the pointer immediately

    // ⚠ What NOT to do:
    // free(p);       // double-free → UB (crash or heap corruption)
    // free(p+1);     // wrong address → UB
    // int x;
    // free(&x);       // not heap memory → UB
    // (never free stack or .data memory)

    return 0;
}
```

Rules for `free`: only free heap memory, only once, using the exact pointer returned by `malloc` / `calloc` / `realloc`.

| The Allocator Under the Hood

What happens when you call `malloc`:

User space: `malloc(100)`



C runtime allocator (e.g. glibc's `ptmalloc`):

Maintains free lists of various sizes
Checks if a suitable free block exists
If not → asks OS for more memory:
 `sbrk()` → extend heap segment
 `mmap()` → map anonymous pages



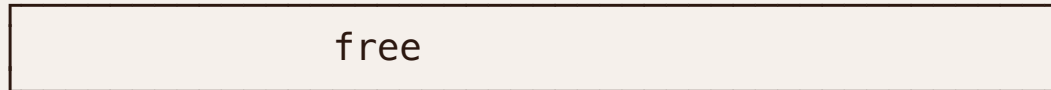
Returns pointer to usable memory
(allocator keeps a header before the block
storing size, flags, etc.)

Each allocation: [header | user data]
 8 bytes `malloc(n)` bytes

Heap Fragmentation

Long-running programs can suffer from **heap fragmentation**:

Initial heap (fresh):



After several malloc/free cycles:



Fragmentation: total free space is large,
but no single contiguous block is big enough
for a new large allocation!

External fragmentation → malloc returns NULL
even when total free > requested size

Long-lived servers (databases, etc.) use custom allocators (jemalloc, tcmalloc) to combat fragmentation.

Dynamic Array Pattern

The idiomatic C pattern for a **growable array**:

```
#include <stdlib.h>
#include <stdio.h>

int main(void) {
    int capacity = 4;
    int size = 0;
    int *data = malloc(capacity * sizeof(int));
    if (!data) return 1;

    for (int i = 0; i < 12; i++) {
        // Grow when full (double the capacity):
        if (size == capacity) {
            capacity *= 2;
            int *tmp = realloc(data, capacity * sizeof(int));
            if (!tmp) { free(data); return 1; }
            data = tmp;
        }
        data[size++] = i * 10;
    }

    for (int i = 0; i < size; i++)
        printf("%d ", data[i]);

    free(data);
    return 0;
}
```

Dynamic 2D Array

Two approaches to allocate a 2D array on the heap:

```
#include <stdlib.h>
#define ROWS 3
#define COLS 4

// Approach 1: flat array, manual index math
int *flat = malloc(ROWS * COLS * sizeof(int));
flat[i * COLS + j] = value;    // access [i][j]
free(flat);

// Approach 2: array of pointers (true 2D)
int **m = malloc(ROWS * sizeof(int *));
for (int i = 0; i < ROWS; i++) {
    m[i] = malloc(COLS * sizeof(int));
}
m[i][j] = value;    // natural notation

// Must free each row, then the pointer array:
for (int i = 0; i < ROWS; i++) free(m[i]);
free(m);
```

*Approach 1 is preferred: **one allocation**, better cache locality, simpler cleanup.*

Returning Heap Memory from Functions

Unlike local variables, heap memory **survives** function return:

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

// Returns heap-allocated copy of a string:
char *duplicate(const char *s) {
    size_t len = strlen(s) + 1;    // +1 for '\0'
    char *copy = malloc(len);
    if (!copy) return NULL;
    memcpy(copy, s, len);
    return copy;    // ✅ caller must free this!
}

int main(void) {
    char *s = duplicate("Hello, world");
    if (!s) return 1;
    printf("%s\n", s);
    free(s);    // caller's responsibility
    s = NULL;
    return 0;
}
```

Ownership & Responsibility

C has no garbage collector — you must track **who owns** each allocation:

Ownership rules (informal contract):

1. The function that malloc's owns the memory unless it explicitly transfers ownership
2. Document who must call free() – in comments
3. Allocator = freer: same component allocs & frees
4. When passing heap pointer to function:
 - does the function take ownership? → it frees
 - or just borrows? → caller frees
5. Use NULL after free to prevent double-free

*In C++/Rust, ownership is enforced by the language. In C, it is enforced by **discipline and documentation**.*

✓ Checkpoint 2 — The Heap

Quick self-check:

1. What is the key difference between `malloc` and `calloc` ?
2. Why must you save `realloc` 's return value in a **temporary** pointer?
3. What are the two ways to create a 2D array on the heap? Which is preferred?
4. Why is it safe to return a heap pointer from a function, but unsafe to return a stack pointer?
5. After `free(p)` , what should you always do to `p` ?

§ 3 — Memory Bugs

Taxonomy of Memory Bugs

Memory Bugs in C

Heap bugs

- Memory leak → allocated but never freed
- Dangling pointer → use after free
- Double free → free the same pointer twice
- Heap buffer overflow → write past allocated block
- Invalid free → freeing wrong address

Stack bugs

- Stack buffer overflow → write past local array
- Dangling stack ptr → return &local_var
- Stack overflow → too-deep recursion / huge locals

General

- Uninitialized read → reading before writing
- Use of NULL ptr → dereferencing NULL
- Aliasing / strict aliasing violations

Memory Leaks

A **memory leak** occurs when allocated memory is never freed:

```
#include <stdlib.h>
#include <stdio.h>

// Leak in a function called many times:
void process_data(int n) {
    int *buf = malloc(n * sizeof(int));
    if (!buf) return;

    // ... do work with buf ...

    // ⚠ BUG: forgot to call free(buf)!
    // Each call to process_data leaks n*4 bytes
}

int main(void) {
    for (int i = 0; i < 1000000; i++) {
        process_data(100);    // leaks 400 bytes × 1M = 400 MB!
    }
    return 0;
}

// Fix: add free(buf) before every return path
```

Detecting Leaks with Valgrind

```
# Compile with debug info:
gcc -g -O0 leak.c -o leak

# Run under Valgrind:
valgrind --leak-check=full --show-leak-kinds=all ./leak
```

```
==12345== HEAP SUMMARY:
==12345==      in use at exit: 400 bytes in 1 blocks
==12345==    total heap usage: 1 allocs, 0 frees, 400 bytes allocated
==12345==
==12345== 400 bytes in 1 blocks are definitely lost in loss record 1 of 1
==12345==    at 0x4C2FB0F: malloc (in /usr/lib/valgrind/...)
==12345==    by 0x10869B: process_data (leak.c:6)
==12345==    by 0x1086C3: main (leak.c:14)
==12345==
==12345== LEAK SUMMARY:
==12345==    definitely lost: 400 bytes in 1 blocks
```

*Valgrind pinpoints the **exact line** where the allocation happened.*

Dangling Pointers (Heap)

A **dangling pointer** points to memory that has been freed:

```
#include <stdlib.h>
#include <stdio.h>

int main(void) {
    int *p = malloc(sizeof(int));
    if (!p) return 1;

    *p = 42;
    printf("*p = %d\n", *p);    // 42

    free(p);

    // ⚠ p is now dangling – the memory may be reused by malloc
    printf("*p = %d\n", *p);    // ⚠ UB: may print 42, garbage, or crash
    *p = 99;                   // ⚠ UB: may corrupt the heap allocator!

    // ✅ Always null after free:
    p = NULL;
    // if (p) *p = 99;    // now safe: NULL check prevents access

    return 0;
}
```

Double Free

Calling `free()` on the same pointer **twice** is Undefined Behavior:

```
#include <stdlib.h>

int main(void) {
    int *p = malloc(sizeof(int) * 10);
    if (!p) return 1;

    free(p);
    free(p);    // ⚠ DOUBLE FREE – heap corruption!
                // May crash immediately, may corrupt heap silently,
                // may be exploitable (heap exploit primitives)

    // ✅ Prevention pattern:
    int *q = malloc(sizeof(int) * 10);
    free(q);
    q = NULL;
    free(q);    // free(NULL) is defined as a no-op – always safe
    return 0;
}
```

`free(NULL)` is **always safe** by the C standard — it does nothing.

Use After Free

Using memory after `free` is one of the most exploited bugs in real software:

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int main(void) {
    char *buf = malloc(16);
    strcpy(buf, "sensitive data");

    free(buf);

    // Allocator may immediately reuse this memory:
    char *new_alloc = malloc(16); // likely gets same address!
    strcpy(new_alloc, "new data");

    printf("%s\n", buf); // ⚠ UAF: prints "new data" (corrupted read)
    strcpy(buf, "hacked"); // ⚠ UAF: corrupts new_alloc's data!

    free(new_alloc);
    return 0;
}
```

Use-after-free is a **critical vulnerability** class (CVE category, exploited in browsers, kernels, etc.).

Heap Buffer Overflow

Writing **past the end** of a heap allocation corrupts allocator metadata:

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

int main(void) {
    char *buf = malloc(8);    // allocate 8 bytes
    if (!buf) return 1;

    // ⚠ Writing 20 bytes into 8-byte buffer:
    memcpy(buf, "Hello, world!!!!", 20);
    // Overwrites:
    //   bytes 0-7:   our buffer (fine)
    //   bytes 8-19:  heap metadata / next chunk header (CORRUPTION)

    // Symptoms: crash later during free(), or silent data corruption

    free(buf);    // may crash here due to corrupted metadata
    return 0;
}
```

Undefined Behavior — What It Really Means

UB is not just "unpredictable output" — it means the **C abstract machine's contract is broken**:

What "undefined behavior" means to the C standard:

- The standard imposes NO requirements on what happens
- The implementation may do ANYTHING

What it means in practice:

- The compiler ASSUMES UB never occurs
- It uses this assumption to optimize
- The resulting code may:
 - Do what you expect (dangerous: masks the bug)
 - Silently corrupt data
 - Crash at a completely unrelated line
 - Be exploited by an attacker
 - Behave differently at -O0 vs -O2

Key insight: "It worked in debug mode" does NOT mean it is correct.

The Compiler & UB — Optimization Example

```
// This looks like it checks for overflow:
int check_overflow(int x) {
    if (x + 1 < x) {          // ← signed overflow is UB
        return 1;           // compiler: "signed overflow is UB,"
    }                       // so x+1 < x can NEVER be true"
    return 0;               // → deletes the entire if branch!
}

// With -O2, the function compiles to:
//   xor eax, eax
//   ret
// (always returns 0, regardless of input)

// Correct version:
#include <limits.h>
int check_overflow_safe(int x) {
    if (x == INT_MAX) return 1; // no UB
    return 0;
}
```

AddressSanitizer in Action

ASan catches bugs at runtime with minimal code changes:

```
gcc -g -fsanitize=address,undefined -o prog prog.c
./prog
```

```
=====
==1234==ERROR: AddressSanitizer: heap-use-after-free on address 0x...
READ of size 4 at 0x602000000010 thread T0
    #0 0x401234 in main prog.c:12
    #1 0x7f... in __libc_start_main
```

```
0x602000000010 is located 0 bytes inside of 16-byte region
[0x602000000010,0x602000000020)
freed by thread T0 here:
    #0 0x... in free
    #1 0x401210 in main prog.c:10
```

```
ASan catches: heap overflow, stack overflow,
               use-after-free, double-free,
               use of uninitialized memory (with -fsanitize=memory)
```

Defensive Patterns for Heap Use

```
#include <stdlib.h>
#include <assert.h>

// 1. Wrapper that never returns NULL:
void *xmalloc(size_t n) {
    void *p = malloc(n);
    if (!p) { perror("malloc"); exit(EXIT_FAILURE); }
    return p;
}

// 2. Safe free macro (nulls the pointer):
#define SAFE_FREE(p) do { free(p); (p) = NULL; } while (0)

// 3. Always validate size before allocating:
void *safe_alloc(size_t count, size_t size) {
    if (count == 0 || size == 0) return NULL;
    if (count > SIZE_MAX / size) return NULL; // overflow check
    return malloc(count * size);
}

// 4. Use: SAFE_FREE(ptr) instead of free(ptr)
```

✓ Checkpoint 3 — Memory Bugs

Quick self-check:

1. What is the difference between a **memory leak** and a **dangling pointer**?
2. Why is `free(NULL)` safe, but `free(p); free(p);` dangerous?
3. What compiler/tool flag catches use-after-free at runtime?
4. Why does "it worked with -O0" not guarantee correctness?
5. What is the **safe free** pattern, and why should you use it?

§ 4 — Preprocessing

The Compilation Pipeline — Overview

```
hello.c
  |
  | Stage 1: Preprocessor (cpp)
  v
hello.i    ← pure C text, no directives
  |
  | Stage 2: Compiler (cc1)
  v
hello.s    ← assembly text
  |
  | Stage 3: Assembler (as)
  v
hello.o    ← object file (binary, ELF on Linux)
  |
  | Stage 4: Linker (ld)
  |   + other .o files + libraries (.a / .so)
  v
hello      ← executable (ELF binary)
```

Run individual stages:

```
gcc -E hello.c -o hello.i    # stop after preprocessing
gcc -S hello.c -o hello.s    # stop after compilation
gcc -c hello.c -o hello.o    # stop after assembly
gcc    hello.c -o hello      # full pipeline
```

Stage 1: Preprocessing

The **preprocessor** is a text transformation step that runs *before* compilation:

What the preprocessor does:

1. `#include` → copy file contents inline
2. `#define` → text substitution
3. `#ifdef` / `#ifndef` → conditional inclusion
4. `#pragma` → compiler hints
5. Strip comments
6. Line number tracking (for error messages)

Input: `hello.c` (C source with directives)

Output: `hello.i` (pure C – no # directives)

The compiler never sees `#include` or `#define` – the preprocessor has resolved them all.

#include — Header Files

`#include` inserts the contents of another file:

```
// System header (searched in system include paths):
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// User header (searched from current directory first):
#include "mymodule.h"
#include "utils/helpers.h"

// What actually happens:
// The preprocessor opens the file and
// pastes its ENTIRE content at this point.

// So this:
#include <stdio.h>
int main(void) { printf("hi\n"); }

// Becomes (simplified):
// [~800 lines of stdio.h content here]
// int main(void) { printf("hi\n"); }
```

Include Guards

Without guards, including a header twice causes **redefinition errors**:

```
// mymodule.h – WITHOUT guard (bad):
struct Point { int x, y; };    // error if included twice!

// mymodule.h – WITH include guard (correct):
#ifndef MYMODULE_H
#define MYMODULE_H

struct Point { int x, y; };
void point_print(struct Point p);

#endif /* MYMODULE_H */

// Alternative: #pragma once (non-standard but widely supported)
#pragma once
struct Point { int x, y; };
```

Rule: Every header file must have an include guard. Name it `FILENAME_H` (uppercase, underscores for dots/slashes).

#define — Object-like Macros

`#define` creates a **text substitution** — it is not a variable:

```
#include <stdio.h>

#define PI          3.14159265358979
#define MAX_SIZE    1024
#define GREETING    "Hello, world"
#define NEWLINE     '\n'

int main(void) {
    double r = 5.0;
    double area = PI * r * r;    // replaced with 3.14159... * r * r
    printf("Area = %f\n", area);

    char buf[MAX_SIZE];         // replaced with char buf[1024]
    printf(GREETING);           // replaced with "Hello, world"
    putchar(NEWLINE);           // replaced with '\n'
    return 0;
}
// No runtime cost: all substitutions happen at compile time
```

#define — Function-like Macros

Macros can take **parameters** and act like inline functions:

```
#include <stdio.h>

#define MAX(a, b)      ((a) > (b) ? (a) : (b))
#define SQUARE(x)      ((x) * (x))
#define ABS(x)         ((x) < 0 ? -(x) : (x))
#define ARRAY_SIZE(a)  (sizeof(a) / sizeof((a)[0]))

int main(void) {
    printf("max(3,7) = %d\n", MAX(3, 7));      // 7
    printf("sq(5)    = %d\n", SQUARE(5));      // 25

    int arr[] = {1, 2, 3, 4, 5};
    printf("size     = %zu\n", ARRAY_SIZE(arr)); // 5

    // ARRAY_SIZE is one of the most useful macros in C
    return 0;
}
```

Macro Pitfalls

Macros are **text substitution** — they don't respect operator precedence:

```
// Missing parentheses around parameters:
#define BAD_SQUARE(x) x * x          // WRONG
#define GOOD_SQUARE(x) ((x) * (x))  // correct

printf("%d\n", BAD_SQUARE(2 + 3));
// Expands to: 2 + 3 * 2 + 3 = 2 + 6 + 3 = 11 (not 25!)

printf("%d\n", GOOD_SQUARE(2 + 3));
// Expands to: ((2 + 3) * (2 + 3)) = 25 ✓

// Double evaluation:
#define BAD_MAX(a, b) ((a) > (b) ? (a) : (b))
int x = 5;
BAD_MAX(x++, 3);
// Expands to: ((x++) > (3) ? (x++) : (3))
// x is incremented TWICE if x > 3!

// Use inline functions instead for anything complex:
static inline int safe_max(int a, int b) { return a > b ? a : b; }
```

#ifdef / #ifndef / #endif

Conditional compilation — include code only under certain conditions:

```
#include <stdio.h>

#define DEBUG_MODE    // comment out to disable debug output

#ifdef DEBUG_MODE
    #define DBG(fmt, ...) fprintf(stderr, "[DBG] " fmt "\n", ##__VA_ARGS__)
#else
    #define DBG(fmt, ...)    // expands to nothing
#endif

int main(void) {
    int x = 42;
    DBG("x = %d", x);    // prints in debug, silent in release

    #ifdef _WIN32
        puts("Running on Windows");
    #elif defined(__linux__)
        puts("Running on Linux");
    #else
        puts("Unknown platform");
    #endif

    return 0;
}
```

Predefined Macros

The compiler defines useful macros automatically:

```
#include <stdio.h>

int main(void) {
    printf("File:      %s\n",    __FILE__);    // source filename
    printf("Line:      %d\n",    __LINE__);    // current line number
    printf("Date:       %s\n",    __DATE__);    // compile date
    printf("Time:       %s\n",    __TIME__);    // compile time
    printf("Function:  %s\n",    __func__);    // current function name (C99)

    // Useful for assertions and logging:
    #define ASSERT(cond) \
        do { \
            if (!(cond)) { \
                fprintf(stderr, "Assert failed: %s, file %s, line %d\n", \
                    #cond, __FILE__, __LINE__); \
                abort(); \
            } \
        } while (0)

    ASSERT(2 + 2 == 4);    // passes
    return 0;
}
```

Inspecting Preprocessor Output

```
# Run only the preprocessor:
gcc -E hello.c -o hello.i

# View what #include <stdio.h> expands to:
gcc -E hello.c | head -50

# Count lines after preprocessing:
gcc -E hello.c | wc -l
# Typically 500-1000+ lines for a simple program!

# Show macros defined:
gcc -E -dM hello.c | grep -i linux

# Example output of -dM:
# #define __linux__ 1
# #define __x86_64__ 1
# #define __GNUC__ 13
# #define __STDC_VERSION__ 201112L
```

Understanding preprocessor output helps debug mysterious macro errors.

✓ Checkpoint 4 — Preprocessing

Quick self-check:

1. What are the **four main tasks** of the C preprocessor?
2. What is the difference between `#include <file>` and `#include "file"` ?
3. What problem do **include guards** solve?
4. Why must macro parameters always be **wrapped in parentheses**?
5. Which flag stops `gcc` after the preprocessing stage?

§ 5 — Compilation & Assembly

Stage 2: Compilation

The **compiler** (e.g., `cc1` inside GCC) translates preprocessed C into assembly:

Input: `hello.i` (pure C text, ~800 lines after preprocessing)
Output: `hello.s` (assembly text, human-readable)

What the compiler does:

1. Lexing → tokenize source into words
2. Parsing → build Abstract Syntax Tree (AST)
3. Semantic → type checking, scope resolution
4. IR gen → translate AST to Intermediate Representation (GIMPLE/LLVM IR)
5. Optimize → constant fold, inline, unroll...
6. Code gen → translate IR to target assembly

The assembly output is specific to the target architecture (x86-64, ARM64, RISC-V...).

Translation Units

Each `.c` file is compiled **independently** as a **translation unit**:

```
project/
├── main.c    → main.o    (one translation unit)
├── utils.c   → utils.o   (one translation unit)
└── math.c    → math.o    (one translation unit)
```

Each `.c` file:

- is preprocessed independently
- is compiled independently
- cannot see the internals of other `.c` files
- can only use symbols declared in headers

Benefits:

- + Incremental compilation (change one `.c` → recompile one `.o`)
- + Encapsulation (static functions/globals are private)
- + Parallel compilation (make -j8)

From C to Assembly

```
# Generate assembly for a C file:
gcc -S -O0 hello.c -o hello.s    # unoptimized, easier to read
gcc -S -O2 hello.c -o hello.s    # optimized

# View the assembly:
cat hello.s
```

```
// Source:
int add(int a, int b) {
    return a + b;
}
```

```
# Assembly output (x86-64, System V ABI):
add:
    push    rbp
    mov     rbp, rsp
    mov     DWORD PTR [rbp-4], edi    # save parameter a
    mov     DWORD PTR [rbp-8], esi    # save parameter b
    mov     edx, DWORD PTR [rbp-4]
    mov     eax, DWORD PTR [rbp-8]
    add     eax, edx                  # eax = a + b
    pop     rbp
    ret                                     # return value in eax
```

| Reading x86-64 Assembly

Key x86-64 instructions you will see in C-generated assembly:

Instruction	Meaning
<code>mov dst, src</code>	Copy value from src to dst
<code>push reg</code>	$RSP -= 8$; $[RSP] = reg$
<code>pop reg</code>	$reg = [RSP]$; $RSP += 8$
<code>add dst, src</code>	$dst = dst + src$
<code>sub dst, src</code>	$dst = dst - src$
<code>cmp a, b</code>	Set flags based on $a - b$
<code>je / jne / jl</code>	Jump if equal / not equal / less
<code>call label</code>	Push return addr, jump to label
<code>ret</code>	Pop return addr, jump to it
<code>lea dst, [...]</code>	Load effective address (pointer arithmetic)

Simple Function in Assembly

```
// C:
int max(int a, int b) {
    if (a > b) return a;
    return b;
}
```

```
# x86-64 assembly (gcc -O1):
max:
    cmp     edi, esi      # compare a (edi) and b (esi)
    jle     .L2           # if a <= b, jump to return b
    mov     eax, edi      # eax = a
    ret

.L2:
    mov     eax, esi      # eax = b
    ret

# Note: no stack frame at -O1 (leaf function optimization)
# Parameters in EDI, ESI (low 32 bits of RDI, RSI)
# Return value in EAX
```

The Intermediate Representation

Modern compilers use an **IR** between C source and assembly:

GCC uses: GIMPLE → RTL → assembly
Clang uses: Clang AST → LLVM IR → assembly

LLVM IR for add(int, int):

```
define i32 @add(i32 %a, i32 %b) {  
entry:  
    %result = add i32 %a, %b    ; typed SSA form  
    ret i32 %result  
}
```

Benefits of IR:

- + Target-independent optimizations
- + Same front-end (C parser) → many backends (x86, ARM, WASM)
- + Static analysis, sanitizers, LTO run on IR

| Compiler Optimizations

Common optimizations performed on the IR:

Optimization	What it does
Constant folding	<code>2 + 3</code> → <code>5</code> at compile time
Dead code elimination	Remove code that never executes
Inlining	Replace function call with function body
Loop unrolling	Expand small loops to reduce branch overhead
Strength reduction	Replace expensive ops: <code>x*2</code> → <code>x<<1</code>
Alias analysis	Determine which pointers can't alias (enables more opts)

```
# Optimization levels:
gcc -O0  # no optimization (fast compilation, debuggable)
gcc -O1  # basic optimizations
gcc -O2  # aggressive, standard for production
gcc -O3  # maximum (may increase binary size)
gcc -Os  # optimize for size
```

Stage 3: Assembler → Object File

The **assembler** (`as`) converts assembly text to binary machine code:

Input: `hello.s` (assembly text)
Output: `hello.o` (ELF object file – binary)

What the assembler does:

1. Parse assembly mnemonics (`mov`, `add`, etc.)
2. Encode each instruction as bytes
(e.g., `"add eax, edx"` → `01 D0`)
3. Record symbol definitions (functions, globals defined in this `.c` file)
4. Record undefined symbol references
(calls to functions in other `.o` files)
5. Produce `.o` file (ELF format on Linux)

Undefined references are left as `PLACEHOLDERS` – the linker will fill them in.

What's Inside an Object File?

ELF Object File (.o) structure:

ELF Header	magic number, architecture, etc.
.text section	machine code (function bodies)
.data section	initialized global variables
.bss section	uninitialized globals (just size)
.rodata section	string literals, const data
Symbol table	defined & referenced symbols
Relocation table	patches needed by linker

Inspecting Object Files with `nm`

```
gcc -c hello.c -o hello.o
```

```
# List symbols in object file:  
nm hello.o
```

```
# nm output:  
0000000000000000 T add          # T = defined in .text (function)  
0000000000000010 T main  
                  U printf      # U = undefined (resolved by linker)  
0000000000000000 D global_x     # D = defined in .data  
0000000000000004 C global_y     # C = common (uninitialized global)
```

```
# Other useful tools:  
objdump -d hello.o             # disassemble  
objdump -s hello.o             # dump all sections  
readelf -h hello.o             # ELF header  
readelf -S hello.o             # section headers  
size    hello.o                # .text / .data / .bss sizes
```

✓ Checkpoint 5 — Compilation & Assembly

Quick self-check:

1. What are the **6 stages** the compiler performs on a translation unit?
2. What flag stops `gcc` after generating **assembly**?
3. Which register holds the **return value** of an integer function on x86-64?
4. What are the four **main sections** of an ELF object file?
5. What does `U` mean in `nm` output?

§ 6 — Linking

Stage 4: Linking

The **linker** (`ld`) combines object files into an executable:

Inputs: `main.o` `utils.o` `math.o` `libc.a`

Linker (`ld`)

Resolve
Relocate
Layout

match undefined refs to definitions
fix placeholder addresses
place sections in final binary

`./hello` (executable ELF)

Symbol Resolution

The linker builds a **global symbol table** and resolves references:

```
main.o defines:    main
main.o needs:      printf, add_numbers

utils.o defines:   add_numbers, multiply
utils.o needs:     (nothing)

libc.a defines:    printf, malloc, free, ...

Resolution:
  main calls add_numbers → found in utils.o  ✓
  main calls printf      → found in libc.a   ✓
  utils multiply         → defined, not called (may be omitted)

Result: all symbols resolved → can produce executable
```

*If a symbol is referenced but **never defined**: linker error — **undefined reference to 'foo'**.*

Undefined vs Defined Symbols — Code Example

```
// file: main.c
extern int shared_value;           // declared, defined elsewhere
void helper(void);                // declared, defined elsewhere

int main(void) {
    helper();
    return shared_value;
}

// file: support.c
int shared_value = 42;             // DEFINITION

void helper(void) {                // DEFINITION
    // ...
}
```

Compile separately:

```
gcc -c main.c -o main.o
```

```
gcc -c support.c -o support.o
```

Link together:

```
gcc main.o support.o -o program # 
```

Without support.o:

```
gcc main.o -o program
```

```
# Error: undefined reference to 'helper'
```

```
# Error: undefined reference to 'shared_value'
```

Relocation

The linker **patches** placeholder addresses in the object file:

In main.o:

```
call 0x00000000 ← placeholder for printf's address  
                (recorded in relocation table)
```

In the final executable:

```
call 0x7f3a2c10 ← printf's actual address (filled in by linker)
```

Relocation entry (in object file):

- offset: where the patch should be applied
- type: what kind of address calculation to use
- symbol: which symbol the address refers to

After relocation:

- All instructions have real addresses
- All global variable references are resolved
- The binary is ready to execute

| The Executable Format (ELF)

ELF (Executable and Linkable Format) is used on Linux:

ELF Executable structure:

ELF Header (64 bytes)	magic 0x7f 'E' 'L' 'F', type, entry point
Program Header Table (for the OS loader)	segments to load into memory
.text segment .rodata segment .data segment .bss segment	executable code read-only data (string literals) initialized globals zero-initialized globals
Section Header Table	for debuggers, linkers (not needed at runtime)

ELF Sections vs Segments

A critical distinction:

SECTIONS (compile/link time view):

`.text, .data, .bss, .rodata, .symtab, .debug_info, ...`

→ Fine-grained, used by linker and debugger

SEGMENTS (runtime view, from Program Header Table):

LOAD segment 1: `.text + .rodata` (R-X: read + execute)

LOAD segment 2: `.data + .bss` (RW-: read + write)

→ Coarser, what the OS actually maps into memory

OS loads segments, not sections:

`mmap()` the LOAD segments into virtual address space

Then jump to entry point (usually `_start` → `main`)

Multiple Translation Units

```
// math.h – shared declarations:
#ifndef MATH_H
#define MATH_H
int add(int a, int b);
int multiply(int a, int b);
#endif

// math.c – definitions:
#include "math.h"
int add(int a, int b) { return a + b; }
int multiply(int a, int b) { return a * b; }

// main.c – uses them:
#include <stdio.h>
#include "math.h"
int main(void) {
    printf("%d\n", add(3, 4));    // 7
    printf("%d\n", multiply(3, 4)); // 12
    return 0;
}
```

```
gcc -c math.c -o math.o
gcc -c main.c -o main.o
gcc main.o math.o -o program
```

Header Files vs Source Files

A common source of confusion for beginners:

Header (.h)	Source (.c)
Declarations	Definitions
<code>int foo(int x);</code>	<code>int foo(int x) { return x+1; }</code>
Struct definitions	Function bodies
Macro definitions	Global variable storage
<code>extern</code> variable declarations	Variable definitions
Included by many files	Compiled once to one <code>.o</code>
No code generated	Produces machine code

Rule: Never put function **definitions** or **variable definitions** in `.h` files (causes multiple definition errors at link time).

Linker Errors vs Compiler Errors

Understanding which stage produced an error:

```
# COMPILER error – problem in one translation unit:
```

```
gcc -c broken.c
```

```
# broken.c:5: error: 'foo' undeclared
```

```
# LINKER error – undefined reference across files:
```

```
gcc main.o utils.o -o program
```

```
# /usr/bin/ld: main.o: undefined reference to `helper'
```

```
# collect2: error: ld returned 1 exit status
```

```
# Linker error causes:
```

```
# – Missing .o file in link command
```

```
# – Typo in function name (declaration ≠ definition)
```

```
# – Missing library (-lm for math, -lpthread for pthreads)
```

```
# – Wrong #ifdef guards excluding the definition
```

```
# Multiple definition linker error:
```

```
# – Function defined in .h (included in multiple .c files)
```

```
# – Global variable defined in .h (not just declared with extern)
```

Inspecting Executables

```
# Disassemble the text section:
objdump -d ./program | head -40

# Show all sections and sizes:
objdump -h ./program

# Show dynamic symbol table:
nm -D ./program

# Show ELF header:
readelf -h ./program

# Show program headers (segments):
readelf -l ./program

# Show shared library dependencies:
ldd ./program

# Show strings in binary (useful for forensics):
strings ./program | head -20
```

| Checkpoint 6 — Linking

Quick self-check:

1. What are the **two main jobs** of the linker?
2. What causes an `undefined reference` linker error?
3. What is the difference between ELF **sections** and **segments**?
4. Why should function **definitions** never go in a `.h` file?
5. What tool shows the symbols defined/undefined in an object file?

§ 7 — Static vs Dynamic Linking

Static Libraries (.a)

A static library (.a) is an archive of object files:

Creation:

```
gcc -c math.c -o math.o
gcc -c stats.c -o stats.o
ar rcs libmymath.a math.o stats.o
```

Contents:

```
ar -t libmymath.a    # list object files inside
nm libmymath.a       # list symbols
```

Linking:

```
gcc main.c -L. -lmymath -o program
# -L.      → search the current directory for libraries
# -lmymath → search for a library named libmymath.a
#
# Alternatively, specify the archive directly:
gcc main.c ./libmymath.a -o program
```

What happens at link time:

- The linker searches for libmymath.a
- It extracts the needed object files from the archive
- Their required code/data are incorporated into the executable
- The .a file is not needed at runtime

Creating and Using a Static Library

```
# Step 1: compile to object files
gcc -c add.c      -o add.o
gcc -c multiply.c -o multiply.o

# Step 2: archive into a static library
ar rcs libcalc.a add.o multiply.o

# Step 3: link with the static library
gcc main.c -L. -lcalc -o program

# -L.      = look for libraries in current directory
# -lcalc   = link with libcalc.a (or libcalc.so if present)

# Step 4: run (no external .a needed – code is inside binary)
./program

# Check: no external dependencies:
ldd ./program
# linux-vdso.so.1 ...
# libc.so.6 ...
# (libcalc is NOT listed – it's baked in)
```

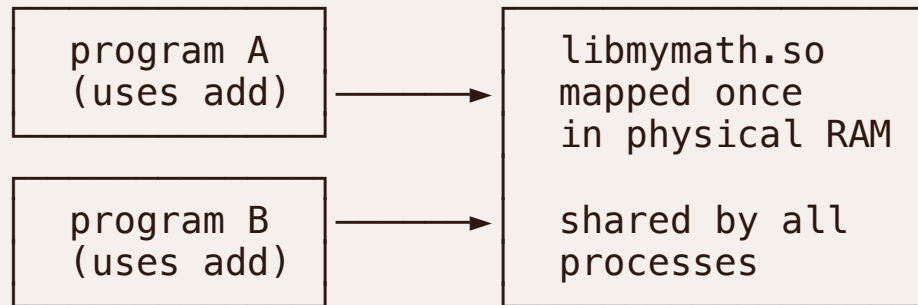
Dynamic/Shared Libraries (.so)

A **shared library** (.so) is loaded at **runtime**, shared between processes:

Creation: compiled with Position-Independent Code (PIC):

```
gcc -fPIC -c math.c    -o math.o
gcc -shared math.o     -o libmymath.so
```

How it works at runtime:



Benefits:

- ✓ Smaller executables (code not duplicated)
- ✓ Library updates without recompiling executables
- ✓ Memory savings (one physical copy for all users)

Creating and Using a Shared Library

```
# Step 1: compile with -fPIC (Position-Independent Code)
gcc -fPIC -c add.c      -o add.o
gcc -fPIC -c multiply.c -o multiply.o

# Step 2: create shared library
gcc -shared add.o multiply.o -o libcalc.so

# Step 3: link (records dependency, does not embed code)
gcc main.c -L. -lcalc -o program

# Step 4: run - dynamic linker must find libcalc.so
export LD_LIBRARY_PATH=. # tell linker where to look
./program

# Or install system-wide:
sudo cp libcalc.so /usr/local/lib/
sudo ldconfig

# Check: dependency is listed:
ldd ./program
# libcalc.so => ./libcalc.so (0x00007f...)
```

Runtime Dynamic Loading

You can load shared libraries **programmatically** at runtime:

```
#include <dlfcn.h>
#include <stdio.h>

int main(void) {
    // Open library at runtime:
    void *handle = dlopen("./libcalc.so", RTLD_LAZY);
    if (!handle) {
        fprintf(stderr, "%s\n", dlerror());
        return 1;
    }

    // Get function pointer by name:
    typedef int (*add_fn)(int, int);
    add_fn add = (add_fn)dlsym(handle, "add");
    if (!add) { fprintf(stderr, "%s\n", dlerror()); return 1; }

    printf("add(3,4) = %d\n", add(3, 4));    // 7

    dlclose(handle);    // release library
    return 0;
}
// Compile: gcc main.c -ldl -o program
```

Static vs Dynamic — Trade-offs

Property	Static (.a)	Dynamic (.so)
Executable size	Larger (library code embedded)	Smaller
Runtime dependencies	No dependency on the library file	Requires .so at runtime
Startup speed	Typically faster	Slightly slower (dynamic linking)
Memory usage	Potentially higher across processes	Potentially lower (shared library pages)
Library updates	Relink application	Replace .so without relinking*
Portability	More self-contained	Depends on compatible runtime libraries
Debugging	Often simpler	Can be more complex
Plugin/module loading	Not applicable to the library itself	✓ Via <code>dlopen()</code>
Version conflicts	Library version fixed in executable	Dependency management required

* Provided the new shared library maintains the **ABI** expected by existing executables.

Rule of thumb: Use static for deployment/embedded; dynamic for large shared libraries (GUI toolkits, system libs).

The Dynamic Linker (ld.so)

The **dynamic linker** (`/lib64/ld-linux-x86-64.so.2`) runs *before* `main`:

Execution of a dynamically linked program:

1. OS loads executable into memory
2. OS sees PT_INTERP segment → path to dynamic linker
3. OS loads the dynamic linker itself
4. Dynamic linker runs (before main!):
 - a. Reads .dynamic section of executable
 - b. Finds all required .so files (DT_NEEDED entries)
 - c. Locates them via:
 - LD_LIBRARY_PATH env variable
 - /etc/ld.so.cache (maintained by ldconfig)
 - /lib, /usr/lib, etc.
 - d. Maps each .so into process address space
 - e. Performs relocations (patches function call addresses)
5. Jumps to `_start` → `main()`

ldd — Inspecting Dependencies

```
# Show shared library dependencies of an executable:
ldd ./program

# Example output:
#   linux-vdso.so.1      (0x00007fffd3c5f7000)  virtual DSO (kernel)
#   libm.so.6            => /lib/x86_64-linux-gnu/libm.so.6
#   libpthread.so.0      => /lib/x86_64-linux-gnu/libpthread.so.0
#   libc.so.6            => /lib/x86_64-linux-gnu/libc.so.6
#   /lib64/ld-linux-x86-64.so.2 (0x00007f3a2c1d0000)

# Check if a library is available:
ldconfig -p | grep libm

# Trace library loading at runtime:
LD_DEBUG=libs ./program 2>&1 | head -30

# Build fully static binary (no .so dependencies at all):
gcc -static main.c -o program_static
ldd program_static
# → "not a dynamic executable"
```

✓ Checkpoint 7 — Static vs Dynamic Linking

Quick self-check:

1. What command creates a static library from object files?
2. Why does shared library compilation require `-fPIC` ?
3. What environment variable tells the dynamic linker where to find `.so` files?
4. What is the role of `ldconfig` ?
5. What function allows loading a shared library **at runtime** from code?

| § Summary: Stack & Heap

Stack:

- Automatic, LIFO, fast, limited size (~8 MB)
- Holds: local variables, parameters, return address, saved registers
- SP moves down on call, up on return — frame destroyed at return
- Never return a pointer to a local variable

Heap:

- Manual (`malloc` / `free`), flexible size, persists across calls
- `malloc` = uninitialized; `calloc` = zero-initialized
- `realloc` may move the block — always use a temporary pointer
- Always `free` exactly once; null the pointer after

§ Summary: Memory Bugs

Bug	Cause	Detection
Memory leak	<code>malloc</code> without matching <code>free</code>	Valgrind
Dangling pointer	Use after <code>free</code> or return of <code>&local</code>	ASan
Double free	<code>free</code> called twice on same ptr	ASan
Heap overflow	Write past <code>malloc</code> 'd block	ASan
Stack overflow	Deep recursion / huge locals	OS signal (SIGSEGV)
Uninitialized read	Read before write	<code>-fsanitize=memory</code>
Wild pointer	Uninitialized pointer deref	ASan

Always compile with: `-Wall -Wextra -fsanitize=address,undefined -g`

§ Summary: Compilation Pipeline

hello.c

gcc -E	Preprocessor	→ hello.i	(text substitution)
gcc -S	Compiler	→ hello.s	(C → assembly)
gcc -c	Assembler	→ hello.o	(asm → machine code + ELF)
gcc	Linker	→ hello	(resolve symbols, relocate)

Key concepts:

- Each .c file = one translation unit (compiled independently)
- Object files contain: .text, .data, .bss, symbol table, relocations
- Linker: resolves undefined symbols, patches addresses, layouts ELF
- Static lib (.a): code embedded in executable at link time
- Dynamic lib (.so): loaded at runtime by ld.so; shared by processes
- dlopen(): load .so programmatically at runtime

Summary

Section	Core Concept
Stack	LIFO, automatic, frames hold locals + return addr + registers
Heap	Manual, <code>malloc</code> / <code>calloc</code> / <code>realloc</code> / <code>free</code> , persists across calls
Memory Bugs	Leaks, dangling, double-free, UAF — use ASan + Valgrind
Preprocessing	Text substitution before compilation; <code>#include</code> , <code>#define</code> , <code>#ifdef</code>
Compilation	C → AST → IR → optimize → assembly; translation units
Assembly	Mnemonics → machine code; ELF object files with sections
Linking	Symbol resolution + relocation → final executable
Static vs Dynamic	<code>.a</code> embeds code; <code>.so</code> shared at runtime via <code>ld.so</code>

| Lecture 1.4 Complete

Recommended Reading

- **K&R** — *The C Programming Language* (Memory management chapters)
- **Bryant & O'Hallaron** — *CS:APP* (Stack frames, linking, ELF)
- **Drepper** — *How to Write Shared Libraries*
- **GCC manual** — gcc.gnu.org
- **Valgrind** — valgrind.org
- **man 3 malloc / man ld.so**

Contact & Q&A

- **Lecturer:** Gabriele Tolomei
- **Email:** tolomei@di.uniroma1.it
- **Office Hours:** Drop me a message, and we'll arrange a time to meet, either in person or online! 😊

Thank you for your attention!